

BL616/BL618

Secure Boot Application Note

Version: 2.0

Copyright @ 2023

www.bouffalolab.com

目录

1 版本信息 4
2 概述 5
3 BL616/BL618 安全特性介绍 6
3.1 Security Boot Chain Of Trust 6
3.2 Security Hardware 7
3.2.1 AES 加解密单元 7
3.2.2 SHA 计算单元 7
3.2.3 PKA 加速单元 7
3.2.4 TRNG 生成单元 7
3.2.5 Flash O(n) T(he) F(ly) CTR/XTS 解密单元 8
3.3 Security Bootrom 8
3.3.1 支持 Bootloader 解密 8
3.3.2 支持 Bootloader 签名验证 8
3.3.3 自保护 8
3.4 Security Bootloader 8
3.4.1 应用程序解密 8
3.4.2 应用程序签名验证 9
3.4.3 Anti-Rollback 9
3.5 Security OTA 9
3.5.1 Anti-Rollback check 10
3.5.2 Signature verification 10
3.5.3 Reboot 10
3.6 Security Storage 10
3.6.1 eFuse 10
3.6.2 Flash 10
3.7 Security Debug 11

4 BL616/BL618 安全启动使用说明 . . . 12
4.1 软硬件准备 . . . 12
4.2 硬件介绍 . . . 12
4.3 编译固件 . . . 13
4.4 部署加密加签服务器 . . . 13
4.5 固件加密 . . . 13
4.5.1 固件加密命令 . . . 14
4.5.2 加密算法 . . . 14
4.5.3 生成的加密固件位置 . . . 14
4.5.4 生成的密钥文件位置 . . . 15
4.5.5 生成的 OTA 文件位置 . . . 15
4.5.6 加密前后固件对比 . . . 16
4.6 固件签名 . . . 17
4.6.1 固件签名命令 . . . 17
4.6.2 生成的签名镜像位置 . . . 18
4.6.3 公钥 Hash(SHA256) 存储位置 . . . 18
4.6.4 生成的 OTA 文件位置 . . . 19
4.6.5 签名前后固件对比 . . . 19
4.7 固件加密 & 签名 . . . 20
4.8 固件烧录 . . . 21
4.9 应用数据加解密 . . . 22
4.9.1 密钥位置 . . . 22
4.9.2 密钥选择 API . . . 24
4.9.3 读写保护 . . . 24
4.10 用户自定义 eFuse 数据 . . . 24
4.10.1 eFuse 使用说明 . . . 24
4.10.2 自定义 eFuse 区域使用接口及注意事项 . . . 30
5 量产烧录 . . . 32
6 关闭 JTAG 调试 . . . 33

1 版本信息

表 1.1: 版本信息

日期	版本	修改内容
2022/7/27	1.0	初版
2023/3/09	2.0	增加对固件加密签名的详细介绍

2 概述

BL616/BL618 是一款支持安全启动和安全应用的智能物联网芯片，本文档主要介绍该芯片具备的安全特性，硬件资源，基本原理及其使用方式。

3 BL616/BL618 安全特性介绍

BL616/BL618 安全框架包括：Security Hardware,Security Bootrom,Security Bootloader,Security OTA,Security Storage 以及 Security Debug.

3.1 Security Boot Chain Of Trust

BL616/BL618 安全启动确保从 Flash 上执行的代码是可信的、安全的并且是被已知主机签名的。如果软件的 Bootloader 或者应用程序被修改，即使一个 Bit 被修改，那么这个软件就被认为是不安全的，并且不会被执行。为了实现这个目标，BL616/BL618 提供了一个从 Bootrom 到 Bootloader 到用户应用程序的安全启动链。

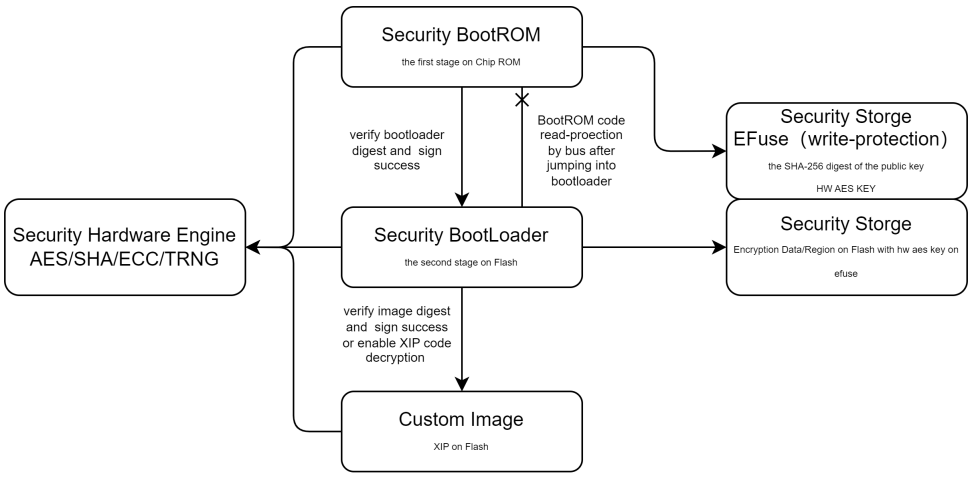


图 3.1: Bouffalo Platform Security

BL616/BL618 支持在 flash 上运行的 Bootloader 和应用固件是加密的，这样可以有效保护客户的代码不会被泄露。当 Flash 加密被使能时，对 Flash 的直接存储访问和执行（XIP）在运行时是透明的，即 Flash 控制器会自动使用 eFuse 中的 AES Key 完成对 flash 上的数据解密。解密密钥可以被保护，只有 Flash 的解密引擎才能使用。

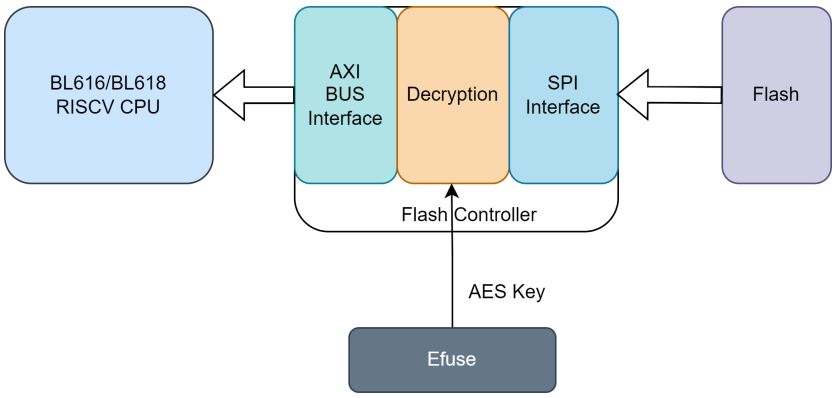


图 3.2: Flash Decrypt Arch

3.2 Security Hardware

为了实现安全目标，BL616/BL618 提供了如下基础安全硬件：

3.2.1 AES 加解密单元

AES 加解密单元支持 ECB/CTR/CBC/XTS 四种加解密模式，密钥长度支持 AES128/256/192。加密的密钥可以从寄存器获取，亦可从芯片内置的 eFuse 获取，以满足用户不同的安全需求。

3.2.2 SHA 计算单元

SHA 计算单元支持 SHA1/SHA-224/SHA-256/SHA-384/SHA-512/SHA-512T224/SHA-512T256，可满足用户安全领域对镜像完整性验证的需求。

3.2.3 PKA 加速单元

PKA 加速单元支持多种大数运算，包括大数乘除法，大数模乘除，大数模幂等，PKA 中大数的长度可灵活配置为 128/256/ 512/ 1024/ 2048 Bits, 配合软件驱动，可以完成 ECC/RSA 的签名，验证等算法，亦可实现 DH/ECDH 等密钥交换算法。

3.2.4 TRNG 生成单元

BL616/BL618 内置的 TRNG 单元能产生真随机数，符合 NIST 800-90A/B 评估标准。

3.2.5 Flash O(n) T(he) F(ly) CTR/XTS 解密单元

BL616/BL618 内置 Flash AES CTR/XTS 解密单元，支持从 Flash 上运行被 AES CTR/XTS 模式加密的固件，内置的硬件解密单元，自动获取解密密钥，自动完成镜像解密，该过程无任何软件参与，密钥长度硬件设计上支持 AES128/192/256，推荐用户选择 AES128 模式。

3.3 Security Bootrom

Bootrom 作为芯片第一阶段的启动程序，提供了信任根（Root of Trust）的启动, 它支持解密二级启动镜像（Bootloader）和验证它的签名。

3.3.1 支持 Bootloader 解密

BL616/BL618 Bootrom 支持启动使用 AES CTR/XTS 加密的二级镜像，密钥长度硬件设计上支持 AES 128/192/256。Bootloader 的加密密钥存在 eFuse 中，在设置读写保护后，只有 Flash 的控制器中的 AES 解密单元能获取这个密钥完成对 Bootloader 的解密，任何软件都无法获取该密钥。

3.3.2 支持 Bootloader 签名验证

BL616/BL618 Bootrom 支持启动使用 ECC 算法签名的二级镜像，能够根据 ROTPK(Root of Trust Public Key) 完成对二级启动镜像的签名验证。ECC 签名验证使用的曲线要求是 SECP256R1，Bootrom 在检测到芯片使能签名后，会先从固件的头部获取镜像签名的公钥，然后计算公钥的的 SHA256, 通过对比存储在 eFuse 中的公钥 Hash 和计算得到的公钥 Hash 相等判断签名主机的合法性。紧接着 Bootrom 读取固件的头部签名值，通过计算镜像的 SHA256 来验证签名的正确性，从而判断镜像的完整性和合法性，确保只有被授权的程序才可运行在芯片上。

3.3.3 自保护

BL616/BL618 的 Bootrom 在启动 Bootloader 以后, 会将自己保护起来，也就是在 Bootloader 阶段是无法对 Bootrom 进行读取或者执行的。

3.4 Security Bootloader

Bootloader 作为芯片第二阶段的启动程序，它支持解密应用程序镜像（APP）和验证它的签名。

3.4.1 应用程序解密

BL616/BL618 Bootloader 支持启动使用 AES CTR/XTS 加密的应用程序镜像，密钥长度硬件设计上支持 AES 128/192/256。应用程序镜像的加密密钥存在 eFuse 中，在设置读写保护后，只有 Flash 的控制器中的 AES 解密单元能获取这个密钥完成对应用程序镜像的解密，任何软件都无法获取该密钥。应用程序使用的加密密钥可以和 Bootloader 使用的加密密钥不同。Bootloader 默认应用程序和 Bootloader 使用相同的密钥，如果应用程序使用的密钥与 Bootloader 不同，则需要在 Bootloader 中修改 Flash 控制器中的解密密钥选择设置。

3.4.2 应用程序签名验证

BL616/BL618 Bootloader 支持应用程序镜像使用 ECC 算法签名验证，同 Bootrom 一样，ECC 使用的曲线要求是 SECP256R1，Bootloader 在检查应用程序镜像完整性、合法性的流程和 Bootrom 是一样的。同样，Bootloader 默认应用程序和 Bootloader 使用相同的公钥签名，如果应用程序使用的公钥与 Bootloader 不同，则需要在 Bootloader 修改应用程序签名公钥 Hash 的读取位置。

3.4.3 Anti-Rollback

防回滚保护功能可确保设备仅执行符合存储在其 eFuse 中的安全版本标准的应用程序。因此，即使应用程序受到信任并由合法密钥签名，它也可能包含一些已撤销的安全功能或凭证，因此设备必须拒绝任何此类应用程序。

BL616/BL618 Bootloader 在启动应用程序镜像的时候，会检查应用程序的版本号，只有版本号大于或者等于 eFuse 中记录的安全版本号时，这个应用程序才会启动。应用程序启动后，会执行一系列诊断操作，根据诊断情况更新 eFuse 中的固件版本信息。

3.5 Security OTA

为了保证 OTA 固件的安全性，应用程序应该使用 HTTPS/MQTTS 等加密链路下载固件。在使能 secure boot 的情况下，OTA 服务器存储的是加密和 (或) 签名后的镜像。

Bouffalo Security OTA 架构如图所示：

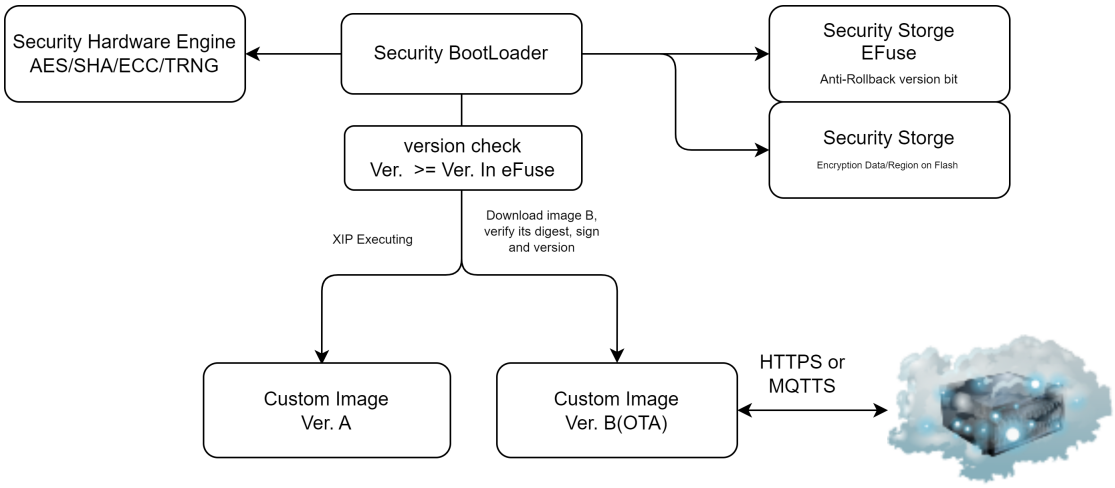


图 3.3: Bouffalo Security OTA

3.5.1 Anti-Rollback check

为了减少不必要的操作，应用固件在进行 OTA 的时候，应先检查要 OTA 固件的版本号是否小于 eFuse 中的安全版本号，如果小于，则可以直接停止 OTA，避免不必要的网络流量浪费。

3.5.2 Signature verification

应用程序在将 OTA 固件写入 Flash 以后，需要对 OTA 的镜像做完整性和合法性验证，如果 OAT 镜像不完整或不合法，则不更新分区表，反之，应用程序应该更新分区表以便通知 Bootloader 启动新的固件。

3.5.3 Reboot

应用程序在确认了 OTA 的镜像完整性和合法性验证，以及更新了分区表之后，调用 Reboot 函数重启系统，Bootrom 按照安全启动流程和规则启动 Bootloader，Bootloader 按照安全启动流程和规则，在确认固件版本号大于等于 eFuse 中的固件版本情况下启动 OTA 后的新固件。新固件启动后，会执行一系列诊断操作，根据诊断情况更新 eFuse 中的固件版本信息。

3.6 Security Storage

3.6.1 eFuse

BL616/BL618 芯片内部带有 4KBits 的 eFuse 存储区域，一方面可以配合 Flash 控制的 AES 解密单元和 Sec Eng 硬件加速引擎完成程序或者数据的加解密，可以配合安全启动模块完成公钥合法性验证，另一方面可以存储用户自定义的数据，扩展 IOT 在安全领域的其它应用。eFuse 主要可以配置存储的内容如下：

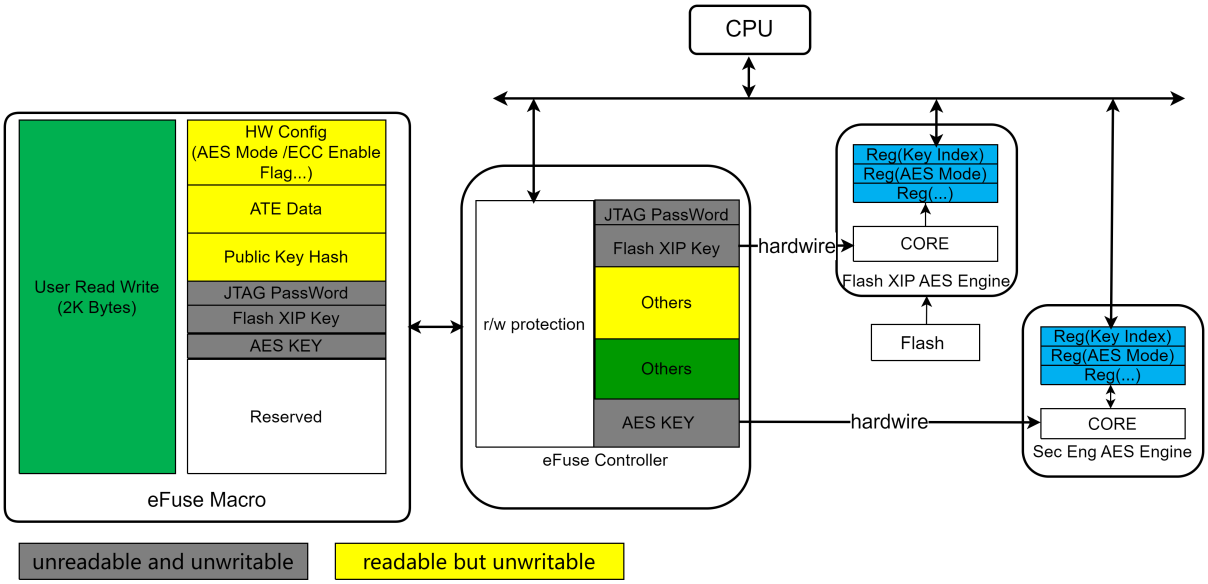
- 镜像签名公钥的 SHA256
- 镜像加密密钥
- 用户数据加解密密钥
- 安全启动配置项
- 用户自定义数据

对于以上存储数据，eFuse 支持对写入的数据开启读写保护，以防止存储的数据被窃取或篡改。例如，对镜像加密密钥，可以开启该密钥的读写保护功能，开启后，该密钥无法被再次被修改，也无法被任何方式读取。

3.6.2 Flash

对于敏感的用户数据，可以将这些数据加密以后存储在 Flash 上，数据加密的密钥烧写在 eFuse 中，这些密钥设置读写保护以后软件无法修改或者读取，只能被硬件加密引擎 Sec Eng 模块用来加密或者解密数据。用户可以通过这种方式实现安全存储。

eFuse 控制器与 Flash XIP 解密单元以及 Sec Eng 加解密引擎单元的关系示意如下：



3.7 Security Debug

为了进一步加强芯片的安全功能，BL616/BL618 支持将芯片的调试模块 JTAG 设置为密码模式或者关闭模式，在密码模式下，用户必须输入正确的密码后才能打开 JTAG 功能。在关闭模式下，用户永远无法使用 JTAG 功能，这种模式可以有效防止固件或者数据被第三方通过 JTAG 调试模块窃取。

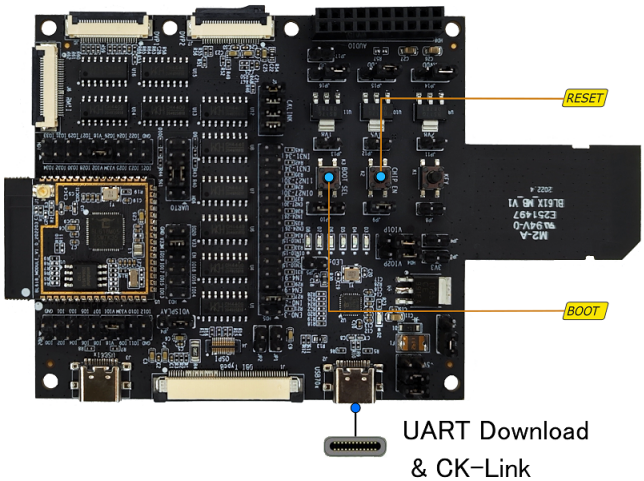
4 BL616/BL618 安全启动使用说明

4.1 软硬件准备

- BL618 开发板，用于验证安全启动的硬件。
- IOT SDK 用于编译生成测试使用的固件。
- 一台服务器，用于部署镜像的加密加签功能。
- bflb_iot_tool，位于 IOT SDK 的 tools/flash_tool 目录下，用于对固件加密，加签或将镜像烧录到 BL616/BL618 上。其中也包含了预编译的 Boot2 二进制文件以及分区表等。

4.2 硬件介绍

BL618 开发板自带一个 ck-link 调试器，这个调试器有两个端口，一个是串口 UART，一个是 JTAG 接口，这两个接口都与 BL616/BL618 连接，可以实现通过串口烧录程序以及通过 JTAG 调试程序。



调试器使用 Type-C 接口与 PC 连接,Type-C 在通信的同时，也给开发板供电, 将 Type-C 接口连接到电脑以后，电脑端会出现一个串口和一个 ck-link 设备。

注解: 如果没有安装 ck-link 驱动，可能不能枚举出 ck-link 设备，在本评估过程中不需要使用 ck-link，只要串口成功枚举即可。

BL618 开发板带有 Boot 选择按键和 Reset 复位按键，芯片主要有两种模式，一种是烧录模式，一种是运行模式。按键使用配置如下：

- 烧录模式: 先按住芯片的 BOOT_SEL 按键，让 Boot 引脚为高电平, 再按下 CHIP_EN/Reset 按键，然后松开 CHIP_EN/Reset 按键，最后松开 BOOT_SEL 按键，这样芯片就进入了烧写模式。
- 运行模式：直接按下 CHIP_EN/Reset 按键，然后松开，芯片就从 Flash 重启。

4.3 编译固件

以 bl616_demo_event 为例，执行以下命令，完成固件编译

```
cd customer_app/bl616_demo_event
./genromap
```

编译完成后，在 build_out 生成了 bl616_demo_event.bin 文件。

4.4 部署加密加签服务器

将 IOT SDK 的 tools/flash_tool 目录拷贝到加密加签服务器，并使用 OpenSSL 产生密钥对。

BL616/618 签名使用的算法是 ECDSA, 椭圆曲线是 SECP256R1, 使用 OpenSSL 产生 ECC 256 密钥对方法如下，用户也可以按照自己的方式产生签名密钥对。

```
openssl ecparam -name secp256r1 -genkey -noout -out privatekey_uecc.pem
openssl ec -in privatekey_uecc.pem -pubout -out publickey_uecc.pem
```

私钥文件 privatekey_uecc.pem 和公钥文件 privatekey_uecc.pem 产生后推荐拷贝到工具的 utils/pem 目录下。

4.5 固件加密

完成固件加密的流程如下：

1. 用户将编译的固件 bl616_demo_event.bin 上传到加密加签服务器。
2. 服务器调用 bflb_iot_tool 命令完成对 Boot2(Bootloader) 和 bl616_demo_event.bin 的加密
3. 用户下载加密以后的固件

4.5.1 固件加密命令

服务器完成对固件加密的命令如下：

```
bflb_iot_tool-ubuntu --chipname=bl616 --boot2="chips/bl616/builtin_imgs/boot2_isp_bl616_v6.5.4/boot2_isp_
↪release.bin" --firmware="bl616_demo_event.bin" --pt="chips/bl616/partition/partition_cfg_4M.toml" --dts=
↪"chips/bl616/device_tree/bl_factory_params_IoTKitA_auto.dts" --key="000102030405060708090A0B0C0D0E0F" -iv=
↪"101112131415161718191A1B00000000" --build
```

其中：

- chipname, 用于设置芯片型号
- boot2, 用于设置使用的 Boot2(Bootloader) 文件
- firmware, 设置加密的应用固件
- pt, 用于设置使用的分区表文件
- dts, 用于设置使用的设备树文件
- key, 用于设置加密使用的 AES 密钥，用户可以根据自己的需要自行产生。
- iv, 用于设置加密使用的 AES IV，用户可以根据自己的需要自行产生。

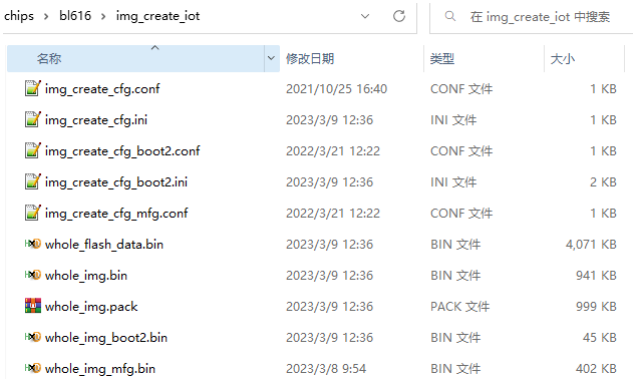
该命令会使用 AES Key 和 AES IV 完成对 Boot2 文件和应用固件镜像的加密。在默认情况下 Boot2 和应用固件使用相同的 AES 密钥和 AES IV。

4.5.2 加密算法

固件加密支持 AES CTR/XTS 模式，密钥长度支持 AES128/256/192，工具默认支持 AES-CTR-128 模式。

4.5.3 生成的加密固件位置

工具生成的加密量产固件保存在工具目录“chips/bl616/img_create_iot”下，如下图所示。

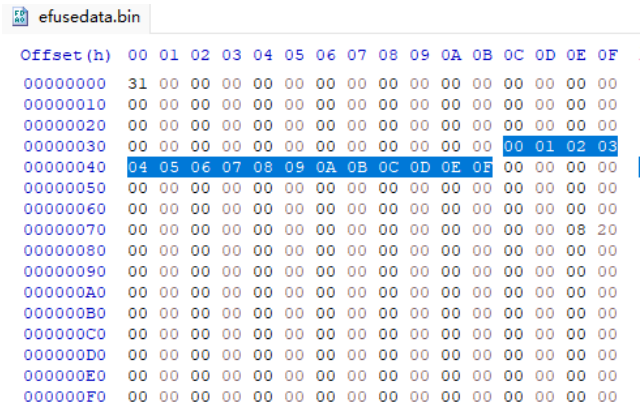


其中，

- whole_img_boot2.bin：是加上启动信息头部的 boot2 文件；
- whole_img.bin：是加上启动信息头部的应用固件；
- whole_flash_data.bin：是将分区表、带启动信息头部的 boot2、带启动信息头部的应用固件、dts 文件等按照在 flash 上地址的排布生成的整体 bin 文件，量产时烧写 flash 可以直接烧写该文件；

4.5.4 生成的密钥文件位置

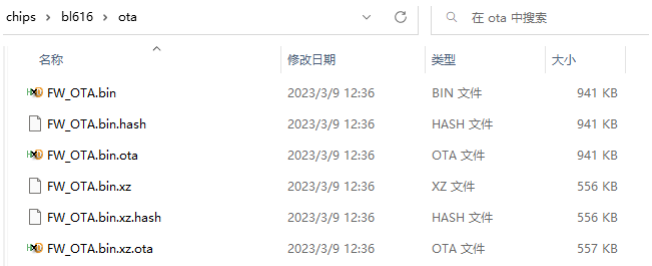
生成的密钥文件 efusedata.bin 保存在“chips/bl616/efuse_bootheader/efusedata.bin”。当选择 AES-CTR-128 模式时，会在 eFuse 中存储长度为 16 字节的密钥（如下图所示在 efusedata 偏移 0x3C 地址的位置存储了 16 字节的密钥），当选择 AES-CTR-256 模式时，会在 eFuse 中存储长度为 32 字节的密钥。为了防止密钥被恶意获取，存储密钥的 eFuse 区域可以设置读写保护。



注解： 工具默认生成的 efusedata.bin 的内容是被加密的 eFuse 数据（这样做的目的也是为了确保 eFuse 数据的安全，博流的烧写工具在接收加密的 efusedata.bin 文件烧写时，会自动解密 eFuse 数据并烧写），用户可通过“chips/bl616/eflash_loader/eflash_loader_cfg.ini”配置文件的“security_save=false”配置项将 eFuse 数据保存成明文，上图展示的是明文的 eFuse 数据。

4.5.5 生成的 OTA 文件位置

同时会在工具目录“chips/bl616/ota”下生成用于 OTA 升级的固件，如下图所示。



其中，

- FW_OTA.bin：是加上启动信息头部的应用固件；
- FW_OTA.bin.hash：是对 FW_OTA.bin 计算 SHA256 并将 SHA256 值加入 FW_OTA.bin 文件末尾生成的带 SHA256 值的文件；
- FW_OTA.bin.ota：是对 FW_OTA.bin 加上 OTA 头部生成的带 OTA 头的文件（非压缩的 OTA 升级使用该文件）；
- FW_OTA.bin.xz：是对 FW_OTA.bin 压缩后生成的压缩文件；
- FW_OTA.bin.xz.hash：是对 FW_OTA.bin.xz 计算 SHA256 并将 SHA256 值加入 FW_OTA.bin.xz 文件末尾生成的带 SHA256 值的文件；
- FW_OTA.bin.xz.ota：是对 FW_OTA.bin.xz 加上 OTA 头部生成的带 OTA 头的文件（压缩的 OTA 升级使用该文件）。

4.5.6 加密前后固件对比

加密的 AES IV 存放在固件的头部，会跟随固件一起烧写到 flash 中。AES IV 在固件头部的存放位置如下图，固件头部偏移 0x100 地址的位置存储了 16 字节的 AES IV。

whole_flash_data.bin

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
0000FFE0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
0000FFF0	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
00010000	42	46	4E	50	01	00	00	00	46	43	46	47	11	00	01	01
00010010	66	99	FF	03	9F	00	B7	E9	04	00	00	01	C7	20	52	D8
00010020	06	02	32	00	0B	01	0B	01	3B	01	BB	00	6B	01	EB	02
00010030	EB	02	02	50	00	01	00	01	01	00	02	01	02	01	AB	01
00010040	05	35	00	00	01	01	00	00	38	FF	FF	F0	77	03	02	40
00010050	77	03	02	F0	2C	01	B0	04	B0	04	32	00	40	0D	14	00
00010060	C8	57	D6	D9	50	43	46	47	04	05	00	00	03	02	01	04
00010070	01	01	01	00	EE	0E	2D	8E	04	01	4C	65	00	10	00	00
00010080	00	00	00	00	B0	82	0C	00	85	94	C9	01	78	EF	BB	16
00010090	C0	4D	D6	20	54	B7	80	CC	0C	5A	01	C5	5F	7A	E0	C2
000100A0	87	04	8F	6B	22	D1	4B	38	01	00	00	00	00	00	00	00
000100B0	00	00	00	A0	00	00	00	00	00	00	00	00	00	00	00	00
000100C0	14	01	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000100D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000100E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
000100F0	00	00	00	00	00	00	00	00	00	00	00	00	1F	EE	B5	AE
00010100	10	11	12	13	14	15	16	17	18	19	1A	1B	00	00	00	00
00010110	49	17	DF	A3	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
00010120	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF
00010130	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF	FF

上图中加密固件分配在 flash 中 0x10000 地址位置，固件的头部（bootheader）存储在 flash 0x10000~0x10FFF 地址区域。

加密前后的固件内容对比如下图，左侧是加密前的固件明文，右侧是加密后的固件密文，最终烧写存储在 flash 上的是右侧的固件密文。



上图中加密固件分配在 flash 中 0x10000 地址位置，其中固件的前 0x1000 区域存储固件的 bootheader，实际的固件内容从 0x11000 地址开始。

4.6 固件签名

完成固件签名的流程如下：

1. 用户将编译的固件 bl616_demo_event.bin 上传到加密加签服务器。
2. 服务器调用 bflb_iot_tool 命令完成对 Boot2(Bootloader) 和 bl616_demo_event.bin 的签名
3. 用户下载签名以后的固件

4.6.1 固件签名命令

服务器完成对固件签名的命令如下：

```
bflb_iot_tool-ubuntu --chipname=bl616 --boot2="chips/bl616/builtin_imgs/boot2_isp_bl616_v6.5.4/boot2_isp_
↪release.bin" --firmware="bl616_demo_event.bin" --pt="chips/bl616/partition/partition_cfg_4M.toml" --dts=
↪"chips/bl616/device_tree/bl_factory_params_IoTKitA_auto.dts" --pk="utils/pem/publickey_uecc.pem" --sk=
↪"utils/pem/privatekey_uecc.pem" --build
```

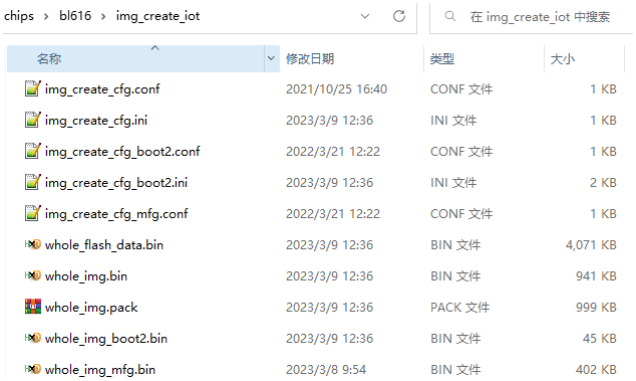
其中：

- chipname, 用于设置芯片型号
- boot2, 用于设置使用的 Boot2(Bootloader) 文件
- firmware, 设置加密的应用固件
- pt, 用于设置使用的分区表文件
- dts, 用于设置使用的设备树文件
- pk, 用于设置签名的公钥，即上文使用 OpenSSL 产生的公钥
- sk, 用于设置签名的私钥，即上文使用 OpenSSL 产生的私钥

该命令会使用 Private Key 和 Public Key 完成对 Boot2 文件和应用固件镜像的签名。在默认情况下 Boot2 和应用固件使用相同的 Private Key 和 Public Key 进行签名。

4.6.2 生成的签名镜像位置

工具生成的签名量产固件保存在工具目录“chips/bl616/img_create_iot”下，如下图所示。

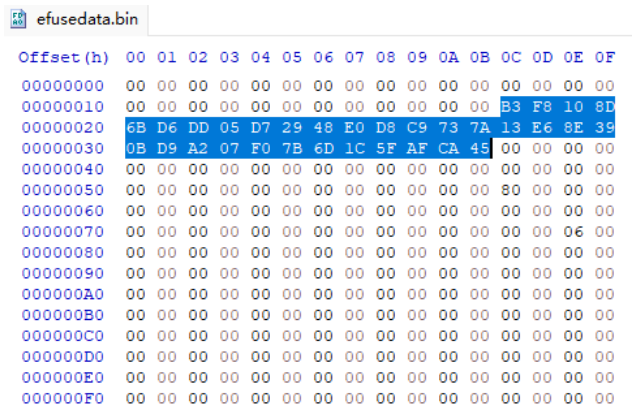


其中，

- whole_img_boot2.bin：是加上启动信息头部的 boot2 文件；
- whole_img.bin：是加上启动信息头部的应用固件；
- whole_flash_data.bin：是将分区表、带启动信息头部的 boot2、带启动信息头部的应用固件、dts 文件等按照在 flash 上地址的排布生成的整体 bin 文件，量产时烧写 flash 可以直接烧写该文件；

4.6.3 公钥 Hash(SHA256) 存储位置

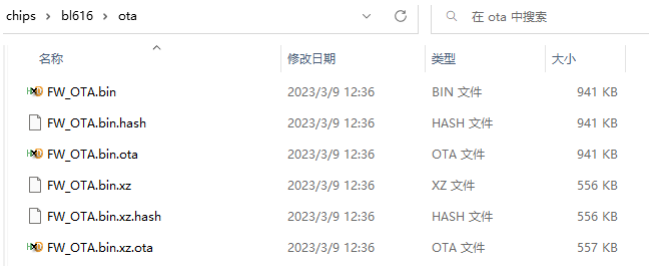
生成的公钥 Hash 文件 efusedata.bin 保存在“chips/bl616/efuse_bootheader/efusedata.bin”。如下图所示在 efuse data 偏移 0x1C 地址的位置存储了 32 字节的公钥 Hash。



注解： 工具默认生成的 efusedata.bin 的内容是被加密的 eFuse 数据（这样做的目的也是为了确保 eFuse 数据的安全，博流的烧写工具在接收加密的 efusedata.bin 文件烧写时，会自动解密 eFuse 数据并烧写），用户可通过“chips/bl616/eflash_loader/eflash_loader_cfg.ini”配置文件的“security_save=false”配置项将 eFuse 数据保存成明文，上图展示的是明文的 eFuse 数据。

4.6.4 生成的 OTA 文件位置

同时会在工具目录“chips/bl616/ota”下生成用于 OTA 升级的固件，如下图所示。

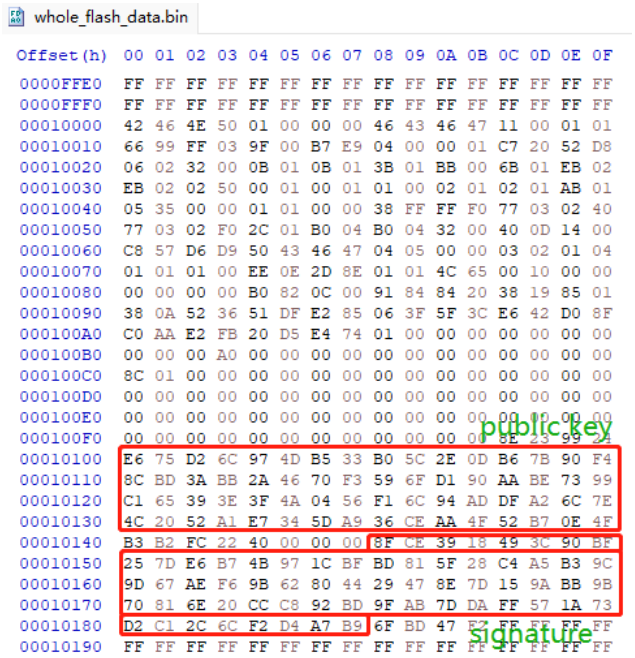


其中，

- FW_OTA.bin：是加上启动信息头部的应用固件；
- FW_OTA.bin.hash:是对 FW_OTA.bin 计算 SHA256 并将 SHA256 值加入 FW_OTA.bin 文件末尾生成的带 SHA256 值的文件；
- FW_OTA.bin.ota：是对 FW_OTA.bin 加上 OTA 头部生成的带 OTA 头的文件（非压缩的 OTA 升级使用该文件）；
- FW_OTA.bin.xz：是对 FW_OTA.bin 压缩后生成的压缩文件；
- FW_OTA.bin.xz.hash：是对 FW_OTA.bin.xz 计算 SHA256 并将 SHA256 值加入 FW_OTA.bin.xz 文件末尾生成的带 SHA256 值的文件；
- FW_OTA.bin.xz.ota：是对 FW_OTA.bin.xz 加上 OTA 头部生成的带 OTA 头的文件（压缩的 OTA 升级使用该文件）。

4.6.5 签名前后固件对比

签名后的固件本身内容没有发生变化，主要是在固件的头部增加了公钥和签名，如下图所示。固件头部偏移 0x100 地址的位置存储了 64 字节的公钥，偏移 0x148 地址的位置存储了 64 字节的签名值。



上图中签名固件分配在 flash 中 0x10000 地址位置，固件的头部（bootheader）存储在 flash 0x10000~0x10FFF 地址区域。

如下图所示，左侧是签名前的固件头部，右侧是签名后的固件头部。

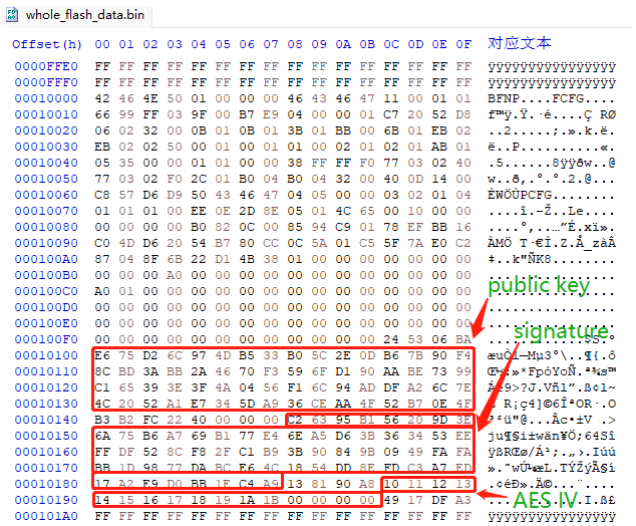


4.7 固件加密 & 签名

当需要将固件同时加密和签名时，执行的命令如下：

```
bflb_iot_tool-ubuntu --chipname=bl616 --boot2="chips/bl616/builtin_imgs/boot2_isp_bl616_v6.5.4/boot2_isp_
↪release.bin" --firmware="bl616_demo_event.bin" --pt="chips/bl616/partition/partition_cfg_4M.toml" --dts=
↪"chips/bl616/device_tree/bl_factory_params_IoTKitA_auto.dts" --key="00010203040506070809 0A0B0C0D0E0F" -iv=
↪"10111213141516171819 1A1B00000000" --pk="utils/pem/publickey_uecc.pem" --sk="utils/pem/privatekey_uecc.pem
↪" --build
```

工具会对镜像先完成加密，然后对加密的固件进行签名。固件签名的公钥、签名值以及固件加密的 AES IV 都存放在固件的头部，当加密和签名同时使能时，加密固件的 AES IV 会存放在签名的公钥和签名值之后。如下图所示：



4.8 固件烧录

1. 从加密加签服务器获取加密和（或）加签后的固件 whole_flash_data.bin 以及存储密钥的 efusedata.bin。
2. 设置芯片进烧录模式

   先按住芯片的 BOOT_SEL 按键，让 Boot 引脚为高电平, 再按下 CHIP_EN/Reset 按键，然后松开 CHIP_EN/Reset 按键，最后松开 BOOT_SEL 按键，这样芯片就进入了烧写模式。
3. 使用如下命令将 whole_flash_data.bin 和 efusedata.bin 分别烧录到 Flash 和芯片的 eFuse 上。

```
bflb_iot_tool-ubuntu --chipname=bl616 --port=/dev/ttyUSB0 --baudrate=2000000 --addr=0x0 --firmware="whole_
↪flash_data.bin" --efuse="efusedata.bin" --single
```

其中：

- chipname, 用于设置芯片型号
- port, 用于设置烧录串口
- baudrate, 设置串口波特率
- addr, 用于设置 flash 烧写起始地址
- firmware, 用于设置 Flash 烧录的文件
- efuse, 用于设置 eFuse 烧录的文件
- single, 用于设置直接下载 Flash 烧录文件，不做额外处理

4. 复位芯片，从 Flash 启动

使用串口工具，打开烧录的串口，波特率选择 2M, 直接按下 CHIP_EN/Reset 按键，然后松开，芯片就会从 Flash 重启, 通过串口工具可以看到 bl616_demo_event 的 log 日志输出，如下图所示。

```
========================================================
################ Hello BL616 FPGA ################
-> Wi-Fi6 Firmware is booting...
Heap[0] [0x230229b0 ~ 0x23030000] size 53 kbytes
Heap[1] [0x62fdfcf0 ~ 0x63010000] size 192 kbytes
rfparam>>pwr_mode is bf
Empty slot:0
rfparam>>tlv wlan pwr_offset[14]: 0,0,0,0,0,0,0,0,0,0,0,0,0,0,
rfparam>>wlan pwr_offset[14]: 0,0,0,0,0,0,0,0,0,0,0,0,0,0,
Empty slot:0
rfparam>>tlv wlan lp pwr_offset[14]: 0,0,0,0,0,0,0,0,0,0,0,0,0,0,
rfparam>>wlan lp pwr_offset[14]: 0,0,0,0,0,0,0,0,0,0,0,0,0,0,
Empty slot:0
rfparam>>tlv bz pwr_offset[5]: 0,0,0,0,0,
rfparam>>bz pwr_offset[5]: 0,0,0,0,0,
rfparam>>pwr_11b[4]: 20,20,20,20,
rfparam>>pwr_11g[8]: 18,18,18,18,18,18,16,16,
rfparam>>pwr_11n_ht20[8]: 18,18,18,18,18,16,15,15,
rfparam>>pwr_11n_ht40[8]: 18,18,18,18,18,16,15,14,
rfparam>>pwr_11ac_vht20[10]: 18,18,18,18,18,16,15,15,15,14,
rfparam>>pwr_11ac_vht40[10]: 18,18,18,18,18,16,15,14,14,13,
rfparam>>pwr_11ac_vht80[10]: 18,18,18,18,18,15,14,13,13,12,
rfparam>>pwr_11ax_he20[12]: 18,18,18,18,18,16,15,15,15,14,13,13,
rfparam>>pwr_11ax_he40[12]: 18,18,18,18,18,16,15,14,14,13,12,12,
rfparam>>pwr_11ax_he80[12]: 18,18,18,18,18,15,14,13,13,12,11,11,
rfparam>>pwr_11ax_he160[12]: 18,18,18,18,18,15,14,13,12,11,10,10,
rfparam>>capcode mode is MF
Empty slot:0
rfparam>>tlv capcode_in 36,capcode_out 36
rfparam>>capcode_in 36,capcode_out 36
rfparam>>tcal.en_tcal = 0
rfparam>>tcal.linear_or_follow = 1
rfparam>>tcal.Tchannels[5]: 2412,2427,2442,2457,2472,
rfparam>>tcal.Tchannel_os[5]: 180,168,163,160,157,
rfparam>>tcal.Tchannel_os_low[5]: 199,186,170,165,160,
rfparam>>tcal.Troom_os = -1
rfparam>>pwr_ble = 13
rfparam>>pwr_bt[3]: 10,8,8,
rfparam>>pwr_zigbee = 13
rfparam>>country_code = 86
```

4.9 应用数据加解密

如果应用程序需要使用 Sen Eng AES 模块对应用数据做加解密，例如从云端获取或者传输到云端的数据进行加解密，可以使用两种模式，一种是临时密钥，一种是固定密钥，如果应用程序使用临时密钥的方式，可以将临时密钥写入 Sen Eng AES 模块的寄存器，然后完成数据的加解密，如果应用程序使用固定密钥方式，并且要求密钥被安全存储，可以将该密钥写入到芯片内置的 eFuse 中，在需要加解密时，设置 Sen Eng AES 模块的加解密密钥来自 eFuse，即可完成使用 eFuse 中的密钥实现数据的加解密。

BL616/BL618 eFuse 中的 Sen Eng 加解密密钥 key 按照 slot 进行组织，每个 slot 的大小为 16 字节，Sen Eng AES 模块提供 key_sel 寄存器，用于选择加解密密钥 key 的 slot 位置。

4.9.1 密钥位置

Sec Eng AES 模块密钥位置如下：

表 4.1: Sec Eng AES 模块密钥位置

Efuse Offset	Nicky Name	Vendor Used	Image Not Encrypt But Sign	Image AES-128 Encrypt & Sign	Image AES-256 Encrypt & Sign
0x00-0x1C		Y			
0x1C-0x2C	Slot0	N	Bootloader Pulic Key Hash	Bootloader Pulic Key Hash	Bootloader Pulic Key Hash
0x2C-0x3C	Slot1	N			
0x3C-0x4C	Slot2	N	Sec Eng AES Key	Bootloader XIP AES Key	Bootloader XIP AES Key
0x4C-0x5C	Slot3	N	Sec Eng AES Key	APP XIP AES Key	
0x5C-0x6C		Y			
0x6C-0x7C	Slot11	Y			
0x7C-0x80		eFuse read write lock			
0x80-0x90	Slot4	N	Sec Eng AES Key	Sec Eng AES Key	APP XIP AES Key
0x90-0xA0	Slot5	N	Sec Eng AES Key	Sec Eng AES Key	
0xA0-0xB0	Slot6	N	Sec Eng AES Key	Sec Eng AES Key	Sec Eng AES Key

表 4.1: Sec Eng AES 模块密钥位置 (continued)

Efuse Offset	Nicky Name	Vendor Used	Image Not Encrypt But Sign	Image AES-128 Encrypt & Sign	Image AES-256 Encrypt & Sign
0xB0-0xC0	Slot7	N	Sec Eng AES Key	Sec Eng AES Key	Sec Eng AES Key
0xC0-0xD0	Slot8	Y			
0xD0-0xE0	Slot9	Y			
0xE0-0xF0	Slot10	Y			
0xF0-0xFC		Y			
0xFC-0x100		eFuse read write lock			
0x100-0x180		N			
0x180-0x1A0		N	Anti-Rollback version	Anti-Rollback version	Anti-Rollback version
0x1A0-0x1B0		N	APP Pulic Key Hash	APP Pulic Key Hash	APP Pulic Key Hash
0x1B0-0X1C0		N			
0x1C0-0X1FC		N			
0x1FC-0x200		eFuse write lock			

Sen Eng AES 模块通过 key_sel 选择使用的密钥：

Key_Sel_1	Key_Sel_0	Image Not Encrypt	Image AES-128 Encrypt	Image AES-256 Encrypt
0	0	{Slot3,Slot2}	{Slot5,Slot4}	{Slot10,Slot6}
	1	{Slot4,Slot3}	{Slot4,Slot5}	{Slot6,Slot10}
	2	{Slot2,Slot4}	{Slot5,Slot4}	{Slot10,Slot6}
	3	{Slot2,Slot4}	{Slot4,Slot5}	{Slot6,Slot10}
1	0	{Slot6,Slot5}	{Slot7,Slot6}	Slot11,Slot7}
	1	{Slot7,Slot6}	{Slot6,Slot7}	{Slot7,Slot11}
	2	{Slot5,Slot7}	{Slot7,Slot6}	{Slot11,Slot7}
	3	{Slot5,Slot7}	{Slot6,Slot7}	{Slot7,Slot11}

详细解释如下：

1. Flash 不加密

如果 Sec Eng 的加解密密钥是 128Bit，并且烧写在 Slot2，则 key_sel_0 需要设置为 2,key_sel_1 设置为 0。

如果 Sec Eng 的加解密密钥是 256Bit，并且烧写在 Slot4，Slot3(字节流第一个字节烧写在 Slot4 的低 8 比特) 则 key_sel_0 需要设置为 1,key_sel_1 设置为 0。

2. Flash 使用 AES-CTR-128 加密

如果 Sec Eng 的加解密密钥是 128Bit，并且烧写在 Slot4，则 key_sel_0 需要设置为 1,key_sel_1 设置为 0。

如果 Sec Eng 的加解密密钥是 256Bit，并且烧写在 Slot5，Slot4(字节流第一个字节烧写在 Slot5 的低 8 比特) 则 key_sel_0 需要设置为 2,key_sel_1 设置为 0。

3. Flash 使用 AES-CTR-256/192 加密

如果 Sec Eng 的加解密密钥是 128Bit，并且烧写在 Slot6，则 key_sel_0 需要设置为 1,key_sel_1 设置为 0。

如果 Sec Eng 的加解密密钥是 256Bit，并且烧写在 Slot10，Slot6(字节流第一个字节烧写在 Slot5 的低 8 比特) 则 key_sel_0l 需要设置为 2,key_sel_1 设置为 0。

4.9.2 密钥选择 API

应用程序为 Sec Eng 选择加密密钥和设置 AES IV 的 API 的原型如下：

```
Sec_Eng_AES_Set_Key_IV_BE(SEC_ENG_AES_ID_Type aesNo,SEC_ENG_AES_Key_Src_Type keySrc,const uint8_t *key,
 ↪const uint8_t *iv)
其中：
aesNo:SEC_ENG_AES_ID0,
keySrc: SEC_ENG_AES_KEY_HW
key:key_sel value pointer
iv:AES IV
```

4.9.3 读写保护

存储在 eFuse 中的 Key，支持读写保护。设置读写保护后，eFuse 中的内容无法被软件修改或者读取。设置读写保护的位置见《eFuse 使用说明》。

4.10 用户自定义 eFuse 数据

4.10.1 eFuse 使用说明

BL616/BL618 带有 4096Bits(512 字节) 的 eFuse，Base Address 为 0x20056000，可以作为 RF 功率校准、频偏校准，用户 MAC 地址或者用户数据的存储区域，同时也是 Flash 上用户代码加密密钥和签名公钥 Hash 的存储区域。每个区域都带有读写保护功能，用户可以根据自身的读写保密需求进行设定。如果不使能芯片的加密功能或者签名功能，那么加密密钥存储位置，签名公钥 SHA256 存储的位置都可以被应用程序使用。

如果芯片使能了加密或者签名功能，那么对应的安全存储区域就不能留给应用程序使用，应用程序需要使用 eFuse 其它区域。整体上，eFuse 功能分配如下，用户可以根据实际需求，选择用作自定义数据的区域。

表 4.3: eFuse 功能分配

Offset	用途	备注
0x00[Bit31:28]	dbg_mode	0011~0001:password mode, 0000:open mode,others:close mode
0x00[Bit27:26]	jtag_dis	如果关闭 JTAG，则该位置 0x3
0x04	debug port password[31:0]	debug port password
0x08	debug port password[63:32]	
0x0C	debug port password[95:64]	
0x10	debug port password[127:96]	
0x14	mac_addr[31:0]	Bouffalo MAC Address(Slot0), 出厂自带的 MAC 地址
0x18	mac_addr_crc[5:0] mac_addr[47:32]	
0x1C	ECC Public Key Hash(SHA256)(slot0)	Bootloader Public Key Hash, 如不使用 ECC 签名功能，该位置可以留给用户自行使用， 该位置可以被设置为写保护
0x20	ECC Public Key Hash(SHA256)(slot0)	
0x24	ECC Public Key Hash(SHA256)(slot0)	
0x28	ECC Public Key Hash(SHA256)(slot0)	
0x2C	ECC Public Key Hash(SHA256)(slot1)	
0x30	ECC Public Key Hash(SHA256)(slot1)	
0x34	ECC Public Key Hash(SHA256)(slot1)	
0x38	ECC Public Key Hash(SHA256)(slot1)	
0x3C	XIP(Sec Eng) AES Key (slot2)	128 Bits Flash AES Key 或者 Sec Eng Key， 如不需要使用, 该位置可以留给用户自行使用， 该位置可以被设置为读写保护， CPU 无法读取或者修改
0x40	XIP(Sec Eng) AES Key (slot2)	
0x44	XIP(Sec Eng) AES Key (slot2)	
0x48	XIP(Sec Eng) AES Key (slot2)	
0x4C	XIP(Sec Eng) AES Key (slot3)	128 Bits Flash AES Key 或者 Sec Eng Key， 如不需要使用, 该位置可以留给用户自行使用， 该位置可以被设置为读写保护， CPU 无法读取或者修改
0x50	XIP(Sec Eng) AES Key (slot3)	
0x54	XIP(Sec Eng) AES Key (slot3)	
0x58	XIP(Sec Eng) AES Key (slot3)	
0x5C	保留	
0x60	保留	
0x64	mac_addr[31:0]	Customer MAC Address1(Slot1)，默认为空， 用户可以在生产环节烧录自己的 MAC
0x68	mac_addr_crc[5:0] mac_addr[47:32]	
0x6C	USB VID[15:0]	USB PID VID
	USB PID[15:0]	
0x70	mac_addr[31:0]	Customer MAC Address1(Slot2)，默认为空， 用户可以在生产环节烧录自己的 MAC
0x74	mac_addr_crc[5:0] mac_addr[47:32]	

表 4.3: eFuse 功能分配 (continued)

Offset	用途	备注
0x78	保留	
0x7C	Read&Write Lock Config	
0x7C[Bit31]	Read Lock 0x6C-0x7C	
0x7C[Bit30]	Read Lock 0x4C-0x5C	
0x7C[Bit29]	Read Lock 0x3C-0x4C	
0x7C[Bit28]	Read Lock 0x2C-0x3C	
0x7C[Bit27]	Read Lock 0x1C-0x2C	
0x7C[Bit26]	Read Lock 0x04-0x14	
0x7C[Bit25]	Write Lock 0x6C-0x7C	
0x7C[Bit24]	Write Lock 0x68-0x6C	
0x7C[Bit23]	Write Lock 0x64-0x68	
0x7C[Bit22]	Write Lock 0x60-0x64	
0x7C[Bit21]	Write Lock 0x5C-0x60	
0x7C[Bit20]	Write Lock 0x4C-0x5C	
0x7C[Bit19]	Write Lock 0x3C-0x4C	
0x7C[Bit18]	Write Lock 0x2C-0x3C	
0x7C[Bit17]	Write Lock 0x1C-0x2C	
0x7C[Bit16]	Write Lock 0x14-0x1C	
0x7C[Bit15]	Write Lock 0x04-0x14	
0x7C[Bit14:0]	保留	
0x80	XIP(Sec Eng) AES Key (slot4)	128 Bits Flash AES Key 或者 Sec Eng Key， 如不需要使用，该位置可以留给用户自行使用， 该位置可以被设置为读写保护， CPU 无法读取或者修改
0x84	XIP(Sec Eng) AES Key (slot4)	
0x88	XIP(Sec Eng) AES Key (slot4)	
0x8C	XIP(Sec Eng) AES Key (slot4)	
0x90	XIP(Sec Eng) AES Key (slot5)	128 Bits Flash AES Key 或者 Sec Eng Key， 如不需要使用，该位置可以留给用户自行使用， 该位置可以被设置为读写保护， CPU 无法读取或者修改
0x94	XIP(Sec Eng) AES Key (slot5)	
0x98	XIP(Sec Eng) AES Key (slot5)	
0x9C	XIP(Sec Eng) AES Key (slot5)	
0xA0	Sec Engine AES Key (slot6)	128 Bits Sec Eng Key, 该位置可以被设置为 读写保护，CPU 无法读取或者修改
0xA4	Sec Engine AES Key (slot6)	
0xA8	Sec Engine AES Key (slot6)	
0xAC	Sec Engine AES Key (slot6)	
0xB0	Sec Engine AES Key (slot7)	128 Bits Sec Eng Key, 该位置可以被设置为 读写保护，CPU 无法读取或者修改
0xB4	Sec Engine AES Key (slot7)	
0xB8	Sec Engine AES Key (slot7)	
0xBC	Sec Engine AES Key (slot7)	

表 4.3: eFuse 功能分配 (continued)

Offset	用途	备注
0xC0	保留	
0xC4	保留	
0xC8	保留	
0xCC	保留	
0xD0	保留	
0xD4	保留	
0xD8	保留	
0xDC	保留	
0xE0	保留	
0xE4	保留	
0xE8	保留	
0xEC	保留	
0xF0	保留	
0xF4	保留	
0xF8	保留	
0xFC	Read&Write Lock Config	
0xFC[Bit31]	Read Lock 0xE0-0xF0	
0xFC[Bit30]	Read Lock 0xD0-0xE0	
0xFC[Bit29]	Read Lock 0xC0-0xE0	
0xFC[Bit28]	Read Lock 0xB0-0xC0	
0xFC[Bit27]	Read Lock 0xA0-0xB0	
0xFC[Bit26]	Read Lock 0x90-0xA0	
0xFC[Bit25]	Read Lock 0x80-0x90	
0xFC[Bit24]	Write Lock 0xF8-0xFC	
0xFC[Bit23]	Write Lock 0xF4-0xF8	
0xFC[Bit22]	Write Lock 0xF0-0xF4	
0xFC[Bit21]	Write Lock 0xE0-0xF0	
0xFC[Bit20]	Write Lock 0xD0-0xE0	
0xFC[Bit19]	Write Lock 0xC0-0xD0	
0xFC[Bit18]	Write Lock 0xB0-0xC0	
0xFC[Bit17]	Write Lock 0xA0-0xB0	
0xFC[Bit16]	Write Lock 0x90-0xA0	
0xFC[Bit15]	Write Lock 0x80-0x90	
0xFC[Bit14:0]	保留	
0x100	ef_zone_00_w0	
0x104	ef_zone_00_w1	
0x108	ef_zone_00_w2	

表 4.3: eFuse 功能分配 (continued)

Offset	用途	备注
0x10C	ef_zone_00_w3	
0x110	ef_zone_01_w0	
0x114	ef_zone_01_w1	
0x118	ef_zone_01_w2	
0x11C	ef_zone_01_w3	
0x120	ef_zone_02_w0	
0x124	ef_zone_02_w1	
0x128	ef_zone_02_w2	
0x12C	ef_zone_02_w3	
0x130	ef_zone_03_w0	
0x134	ef_zone_03_w1	
0x138	ef_zone_03_w2	
0x13C	ef_zone_03_w3	
0x140	ef_zone_04_w0	
0x144	ef_zone_04_w1	
0x148	ef_zone_04_w2	
0x14C	ef_zone_04_w3	
0x150	ef_zone_05_w0	
0x154	ef_zone_05_w1	
0x158	ef_zone_05_w2	
0x15C	ef_zone_05_w3	
0x160	ef_zone_06_w0	
0x164	ef_zone_06_w1	
0x168	ef_zone_06_w2	
0x16C	ef_zone_06_w3	
0x170	ef_zone_07_w0	
0x174	ef_zone_07_w1	
0x178	ef_zone_07_w2	
0x17C	ef_zone_07_w3	
0x180	ef_zone_08_w0	Anti-Rollback version
0x184	ef_zone_08_w1	
0x188	ef_zone_08_w2	
0x18C	ef_zone_08_w3	
0x190	ef_zone_09_w0	
0x194	ef_zone_09_w1	
0x198	ef_zone_09_w2	

表 4.3: eFuse 功能分配 (continued)

Offset	用途	备注
0x19C	ef_zone_09_w3	Anti-Rollback version
0x1A0	ECC Public Key Hash(SHA256)	App Public Key Hash 如不使用 ECC 签名功能， 该位置可以留给用户自行使用，该位置可以被设置为写保护
0x1A4	ECC Public Key Hash(SHA256)	
0x1A8	ECC Public Key Hash(SHA256)	
0x1AC	ECC Public Key Hash(SHA256)	
0x1B0	ECC Public Key Hash(SHA256)	
0x1B4	ECC Public Key Hash(SHA256)	
0x1B8	ECC Public Key Hash(SHA256)	
0x1BC	ECC Public Key Hash(SHA256)	
0x1C0	ef_zone_12_w0	
0x1C4	ef_zone_12_w1	
0x1C8	ef_zone_12_w2	
0x1CC	ef_zone_12_w3	
0x1D0	ef_zone_13_w0	
0x1D4	ef_zone_13_w1	
0x1D8	ef_zone_13_w2	
0x1DC	ef_zone_13_w3	
0x1E0	ef_zone_14_w0	
0x1E4	ef_zone_14_w1	
0x1E8	ef_zone_14_w2	
0x1EC	ef_zone_14_w3	
0x1F0	ef_zone_15_w0	
0x1F4	ef_zone_15_w1	
0x1F8	ef_zone_15_w2	
0x1FC	Write Lock Config	
0x1FC[Bit31]	Write Lock 0x1F0-0x1FE	
0x1FC[Bit30]	Write Lock 0x1E0-0x1F0	
0x1FC[Bit29]	Write Lock 0x1D0-0x1E0	
0x1FC[Bit28]	Write Lock 0x1C0-0x1D0	
0x1FC[Bit27]	Write Lock 0x1B0-0x1C0	
0x1FC[Bit26]	Write Lock 0x1A0-0x1B0	
0x1FC[Bit25]	Write Lock 0x190-0x1A0	
0x1FC[Bit24]	Write Lock 0x180-0x190	
0x1FC[Bit23]	Write Lock 0x170-0x180	
0x1FC[Bit22]	Write Lock 0x160-0x170	
0x1FC[Bit21]	Write Lock 0x150-0x160	
0x1FC[Bit20]	Write Lock 0x140-0x150	

表 4.3: eFuse 功能分配 (continued)

Offset	用途	备注
0x1FC[Bit19]	Write Lock 0x130-0x140	
0x1FC[Bit18]	Write Lock 0x120-0x130	
0x1FC[Bit17]	Write Lock 0x110-0x120	
0x1FC[Bit16]	Write Lock 0x100-0x110	
0x1FC[Bit15:0]	ef_zone_15_w3	

在该表格中，RF 功率校准，频偏校准等参数的写入，需要配合 RF 测试固件和测试仪器一起使用，因此在这里不做介绍，对应位置被标记为“保留”。其它功能位置如表格中所示，主要分为 MAC 地址存储区，ECC 签名公钥 Hash 存储区，Flash 加密密钥存储区，加密引擎硬件密钥存储区，客户自定义的参数存储区。如果 Customer MAC Address 地址区域不使用，可以把这个区域用于用户自定义参数存储区。

如果用户不使用安全启动功能，则 XIP(Flash) AES Key 区域和 Sec Engine AES Key 区域可用于用户自定义参数存储区。

地址 0x1C-0x3C 共 32 字节 256Bits，用来存储 ECC 签名的公钥 Hash，使用的是 SHA256, 在使能 ECC 签名的时候，Bootrom 会读取该位置验证公钥的合法性，然后进行签名的验证，当使能 ECC 签名的时候，需要在此位置烧录签名公钥的 Hash，并且该位置只能设置写保护，不能设置读保护，因为该位置需要被 Bootrom 读取，如果不使能 ECC 签名，则该位置可以用作用户参数存储区。

地址 0x3C-0x5C 是预留 Flash AES 加密密钥，如果用户不使能 Flash 用户程序加密功能，则地址 0x3C-0x5C 可以用作客户自定义的参数存储区。

地址 0x80-0xC0 是预留 Sec Engine AES 加密密钥，如果用户不需要使用 eFuse 中的加密密钥，则地址 0x80-0xC0 可以用作客户自定义的参数存储区。

4.10.2 自定义 eFuse 区域使用接口及注意事项

eFuse 由于其自身的特点，因此在读写操作上有一些事项需要留意。

1. 标准驱动 bl616_ef_ctrl.c 中提供了用户自定义区域数据读写的统一接口。
2. eFuse 读写函数的参数 addr/offset 和 data/pword 指针须是 4 字节对齐。
3. 烧写 eFuse

```
void EF_Ctrl_Program_Direct(uint32_t offset, uint32_t *pword, uint32_t count)
```

该函数主要用于将用户数据烧写到对应的 eFuse 寄存器, 需要注意的是,offset 和 pword 必须是 4 字节对齐,count 表示要烧写的 word 个数。

```
入口参数：
offset: offset of eFuse address to program
pword: data pointer to buffer which is aligned to word
count: count of data in words to program
```

例如, 要从 0x1C 地址处烧写 8 个字节，函数调用如下 EF_Ctrl_Program_Direct(0x1C,buf,2)。

4. 读取 eFuse

```
void EF_Ctrl_Read_Direct(uint32_t offset, uint32_t *pword, uint32_t count)
```

该函数主要用于从 eFuse 中读取已经烧录的数据。需要注意的是,offset 和 pword 必须是 4 字节对齐,count 表示要读取的 word 个数。

```
入口参数：
offset: offset of eFuse address to read
pword: data pointer to buffer which is aligned to word
count: count of data in words to read
```

例如, 要从 0x1C 地址处读取 8 个字节，函数调用如下 EF_Ctrl_Read_Direct(0x1C,buf,2)。对应读保护的区域进行读操作，得到的结果将是 0。

5 量产烧录

客户在量产环节，只要将“bl616/img_create_iot”目录下的生产烧录文件包 whole_img.pack(包含加密加签后的固件以及对于的密钥文件) 给到批量烧写工具，使用批量烧写工具的” 开发模式”，导入 whole_img.pack 烧录包。烧录工具可自行完成加密镜像的烧写和解密密钥的烧写。

6 关闭 JTAG 调试

BL616/BL618 支持关闭 JTAG 模块，关闭的方式是通过配置 eFuse 中的控制位 jtag_dis，如果用户需要关闭 JTAG 模块，只需要在烧写工具的配置文件中将 jtag_dis 设为 3，然后将生成的 efusedata.bin 文件烧录到芯片的 eFuse 中，重启芯片后，即可将芯片的 JTAG 功能永久关闭。